AEP as the Agentic-Container OS

High-Level Design (HLD) · v1.1 · Status: Proposed · Branch: `docs/dual-lane-governance-proposal` · Companion: `aep-lld-v1.1.html`
Supersedes: v1 (dual-lane governance proposal, `docs/draft-proposal.html` v1)

1. Purpose and scope

This document consolidates the design decisions reached in the working discussion into one reference: how the Autonomous Engineering Platform (AEP) is built and deployed portably, and how it governs two kinds of autonomous execution — fast local ralph-loop lanes and sandboxed ESF (Engineering Software Factory) pipelines — on a shared set of core services.

It is a design, not a record of finished work. Section 9 states exactly what exists in the repository today and what is still to be built; reading the rest of the document as "already implemented" would be wrong.

2. Decision log

#	Decision	Rationale	Status
D1	Orchestration tier is a single Go binary, `aep`; NeuroStrata stays in Rust.	Static cross-compiled binaries need no Python on Windows/embedded hosts; `golang.org/x/sys` gives Job Objects, `flock` / `LockFileEx` and process groups. Python's one real edge (`ast`-based test classification in `mechanical_verify.py`) is narrow.	BUILT (CORE)
D2	Artifact targets: `dist/bin/aep-{linux,darwin}-{amd64,arm64}`, `dist/vm/linux-{amd64,arm64}`, plus OCI image/context.	Covers Linux servers, the 8 GB Muse ARM64 VM, Apple Silicon and Intel Macs, and any Docker/Podman host. Native Windows is explicitly out of the target list (WSL2 or container only).	BUILT
D3	Two execution lanes: Tier 1 ralph-loop, Tier 2 ESF.	Local worktree lanes are fast but share the host kernel; ESF adds microVM isolation and durable workflows for untrusted or high-blast-radius work.	PLANNED
D4	AEP is the "container OS": five shared core services on the substrate; lanes are ephemeral containers above it.	One memory, one ledger, one rate limiter, one policy for every lane — no per-lane drift.	PLANNED
D5	Runtime is host-family dependent: on macOS, *one large shared Colima VM* (`vz` + `virtiofs`, sized 12 CPU / 48 GB / 220 GB) with Docker/Podman containers spawned on demand inside it; on Linux, Podman/Docker directly on host. Never one VM per lane in either family.	Container spawning is need-based and admission-bounded inside the single VM/host; per-lane VMs only where kernel-level (microVM) isolation is required, i.e. Tier 2 on `linux-native` with KVM.	PLANNED
D6	`beads` is the single macro task ledger. The Superpowers SDD ledger (`.superpowers/sdd/<plan>/progress.md`) stays a per-plan micro ledger and does not replace it.	Different lifetimes and purposes: durable cross-session backlog vs. ephemeral compaction-recovery record.	PLANNED
D7	ESF workflow IDs are keyed to beads issue IDs (`esf-beads-<ID>`).	Prevents orphan factory runs and dual-backlog drift.	PLANNED
D8	Factory-tagged tasks degrade explicitly when Temporal/CubeSandbox are unreachable: quarantine by default. A downgrade to a fenced local lane requires the explicit authenticated `--override-tier1-risk-reassessment --reason="..."` path and a decision record with operator identity — never a silent or implicit downgrade.	Removes the load-bearing failure found in review: the loop stalling on infrastructure that does not exist on most hosts, without weakening the isolation boundary.	PLANNED
D9	Local-first pluggable Tier-2 sandbox fabric: CubeSandbox (Linux hosts with `/dev/kvm`), `libkrun` (macOS native Hypervisor/Virtualization.framework), and `gVisor`	Breaks the hard dependency on Linux nested KVM: macOS runs native microVMs via `libkrun`, while KVM-free edge/cloud nodes isolate	PLANNED

	(`runsc` in `systrap` mode for KVM-free Linux/cloud). Remote sandbox clusters are strictly an emergency fallback of last resort if all local sandbox runtimes fail.	untrusted tasks via `gVisor` user-space kernel. Update 2026-11-09: nested KVM inside Colima is also *confirmed* (`--nested-virtualization` exposes `/dev/kvm` on M3 Max / Colima 0.10.3), so CubeSandbox-in-Colima is a valid macOS Tier-2 route alongside native `libkrun` .	

3. Goals and non-goals

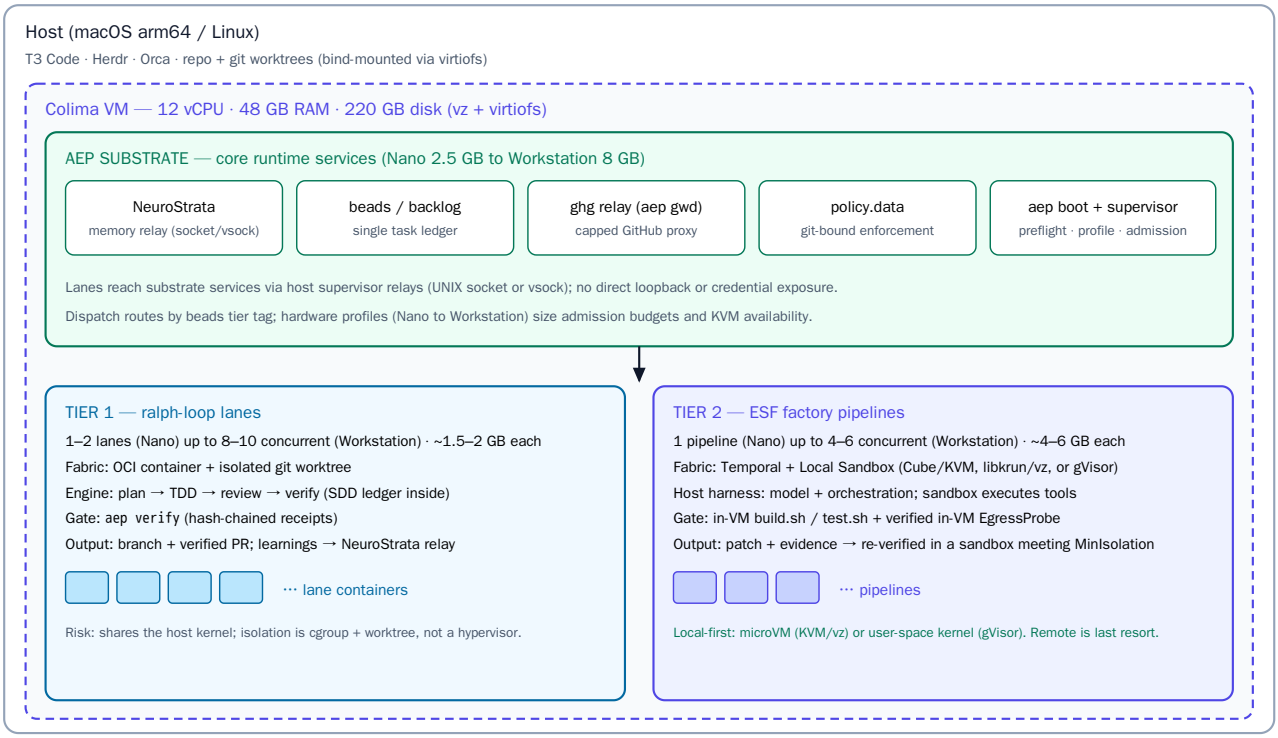
Goals

- One-command install and run on Linux (x86_64 and ARM64), the Muse 8 GB VM, and macOS (Intel and Apple Silicon).
- Many parallel lanes under one governance plane with bounded, predictable resource use.
- Deterministic verification decides success; an agent saying "done" never does.
- Pluggable sandbox isolation for Tier 2: hardware microVMs via CubeSandbox/libkrun, or user-space kernel sandboxing via gVisor on KVM-free hosts.
- Fail closed: no autonomous run starts if the environment is unfit.

Non-goals (v1.1)

- Native Windows binaries or service wrappers.
- macOS guest VM images (macOS runs the native binary under `launchd`).
- Replacing `beads` or NeuroStrata.
- Modifying the vendored `reference/esf` tree (it changes only by re-vendoring).

4. System architecture



5. Components

5.1 Core runtime services (substrate)

Service	Responsibility	Key constraint
NeuroStrata	Hybrid vector + full-text memory (`ladybug.db` , nomic-embed-text-v1.5). Inscription and search via host-mediated socket relay (`aep memoryd`) or vsock outbox.	Host mode: loopback `127.0.0.1:34343` private to host. Compose mode: `neurostrata:34343` on the private `aep-net` network, with no host port published. Lanes connect only via the UNIX socket; embedding cache pre-seeded; `HF_HUB_OFFLINE=1` ; DB backed up by change-detected watchdog.
beads / backlog	Single canonical task ledger; carries the tier tag and the foreign key used by ESF workflow IDs.	Telemetry off (`bd metrics off`); Dolt remote host allowlisted if egress is blocked.
ghg	Counted, coalescing GitHub gateway: served to lanes via `aep gwd` IPC relay; 3,500/5,000 core req/h self-cap, one poller per watched run.	Lanes communicate via client shim to relay socket; GitHub PAT credentials remain strictly on host.
policy.data	Env-variable scrub fragments, sensitive-path patterns, hardware profile; bound to the committed git blob.	Edits land via PR; boot refuses a policy that differs from `HEAD` .
aep boot / supervisor	Fail-closed preflight, dispatch, capacity admission, profile auto-detection.	Exit non-zero rather than start in a degraded state.

5.2 Execution lanes

	Tier 1 · ralph-loop	Tier 2 · ESF
Intended work	Features, fixes, refactors, tests, docs	Untrusted dependencies, CVE remediation, multi-repo or high-blast-radius change
Isolation	Container + git worktree + process group / Job Object	Pluggable local-first sandbox: Hardware microVM (CubeSandbox on Linux KVM, libkrun on macOS vz) OR User-space kernel (gVisor runsc systrap on KVM-free hosts) per run, destroyed on exit. Remote cluster is strictly an emergency last resort.
Model	The trusted host harness runs the agent loop and all model inference outside the lane, and selects the model tier. A lane holds no provider credential and cannot originate a model request; lane output is untrusted tool data.	
Orchestration	Scheduler tick (default 2 h), ≤6 iterations, 3-strike stop; host harness orchestrates, lane executes allowlisted argv over `exec.sock`	Temporal durable workflow with replay-safe activities
Verification	Host harness runs `aep verify`; static checks only in Tier 1	Repo `build.sh` / `test.sh` in a fresh sandbox meeting the task `Requirement`
Human gate	PR review; circuit breaker	`factory review --approve/--reject`

6. Scalable resource model (Nano 2vCPU / 8GB to Workstation 12vCPU / 48GB)

Profiles are two-axis — Host Family × Size Tier. Host families select the local sandbox engine: `linux-kvm` (CubeSandbox microVM via `/dev/kvm`), `mac-native-vz` (hardware microVM via `libkrun` / Apple `Virtualization.framework`), or `linux-gvisor` (user-space kernel via `runsc systrap` without KVM). Size tiers set admission budgets:

Profile	Target hardware	Allocation	Substrate	Tier 1 (Ralph)	Tier 2 (ESF Local Backend)	Isolation Mechanism
`nano-8g`	Intel N100, RPi 5, Cloud 2vCPU/8GB	2 vCPU · 8 GB · 30 GB	2560 MiB (2.5 GiB) · 10 GB	3584 MiB (3.5 GiB); 1 lane @ 2 GiB + 1.5 GiB headroom	1536 MiB (1.5 GiB); 1 lane via gVisor systrap	User-space kernel (no KVM needed)
`small-16g`	Intel Core i5, AMD Ryzen 5, Cloud 4vCPU/16GB	4 vCPU · 16 GB · 80 GB	1 vCPU · 3.5 GB · 15 GB	6.5 GB budget (2–3 lanes @ 2GB)	6.0 GB budget (1 pipeline @ 6GB)	Cube/KVM (Linux) or gVisor (Cloud)
`medium-32g`	Intel Core i7/i9, AMD Ryzen 7/9, Cloud 8vCPU/32GB	8 vCPU · 32 GB · 150 GB	2 vCPU · 4.5 GB · 25 GB	12.0 GB budget (4–6 lanes @ 2GB)	15.0 GB budget (2–3 pipelines @ 5GB)	Cube/KVM (Linux) or libkrun (macOS)
`workstation-48g`	Apple Silicon M3 Max / Linux Threadripper	12 vCPU · 48 GB · 220 GB	2 vCPU · 8.0 GB · 30 GB	16.0 GB budget (8–10 lanes @ 1.5–2GB)	24.0 GB budget (4–6 pipelines @ 4–6GB)	libkrun/vz (macOS) or Cube/KVM (Linux)

> Nested-virtualization evidence (2026-11-09).
> Confirmed 2026-11-09 on Apple M3 Max (macOS 27.2, Colima 0.10.3, `vz` + `virtiofs`): `colima start --nested-virtualization` exposes `/dev/kvm` inside the VM (and in `--privileged` containers); QEMU `accel=kvm` is accepted for a nested L2 guest; Docker 29.5.2 containers run in-VM; Temporal (sqlite dev server) serves UI on `:8233`. Without `--nested-virtualization`, `/dev/kvm` is absent — the flag is required. CubeSandbox itself was not installed/tested.

> Local-first isolation hierarchy.
> AEP prioritizes on-node isolation: (1) native hardware microVMs via CubeSandbox/KVM on Linux or `libkrun` / `vz` on macOS; (2) user-space kernel isolation via `gVisor` (`runsc` in `systrap` mode) when hardware virtualization is absent (e.g. Intel N100 or non-metal Cloud VMs). Remote sandbox clusters are strictly an emergency fallback of last resort if all local sandbox runtimes fail.

7. Platform and artifact coverage

Artifact	Covers	Does not cover
`dist/bin/aep-linux-amd64`	Linux x86_64 servers, cloud VMs, CI runners	32-bit x86; native Windows
`dist/bin/aep-linux-arm64`	Muse ARM64 VM, Graviton/Ampere, Raspberry Pi 4/5 (64-bit)	32-bit ARM
`dist/bin/aep-darwin-arm64`	Apple Silicon Macs	Intel Macs (Rosetta only)
`dist/bin/aep-darwin-amd64`	Intel Macs	—
`dist/vm/linux-{amd64,arm64}`	Cloud-init + systemd bundles with per-arch hardware profile	Non-systemd distros
OCI image / `dist/oci` context	Any Docker/Podman host; Kubernetes-style schedulers	Hosts with no container runtime

8. Dispatch and lifecycle

1. Intake. Work is a beads issue carrying a tier tag (`local` or `factory`) and acceptance criteria.
2. Preflight. `aep boot` must pass: binaries, NeuroStrata answering, standing guards present, policy blob matches `HEAD` , memory floor and hardware profile met.
3. Admission. Dispatcher reserves RAM/CPU for the lane against the active profile budget; if it cannot, the task waits (bounded) rather than oversubscribing.
4. Execute. The trusted host harness runs plan/TDD/review/model inference. Tier 1: claim worktree, start the lane executor with `memory.sock` , `ghg.sock` , `exec.sock` , and `git.sock` ; the lane runs allowlisted commands and commits through `git.sock` with no host `.git` mount. Tier 2: submit `factory run` with workflow ID `esf-beads-<ID>` ; the sandbox executes tools, never the model loop.
5. Verify. Both lanes emit signed receipts. For Tier-2 factory work containing untrusted code, independent patch re-verification runs in a fresh, disposable sandbox whose isolation meets the task `MinIsolation` , with closed egress—never directly on the shared Tier-1 host kernel. A host that cannot meet the floor quarantines before the original run.
6. Close. PR merges, `bd close` , learnings inscribed to NeuroStrata via supervisor relay, lane and reservation released.

9. Governance invariants

- No lane starts unless `aep boot` passes (fail closed).
- All GitHub traffic from lanes goes through `ghg` via the `aep gwd` IPC relay. Lane containers hold no GitHub credentials and cannot connect directly to api.github.com.
- `policy.data` must equal the committed blob; lanes run with credential-named env vars scrubbed.
- ESF runs require `hardening.acknowledge_open_egress = false` (the upstream default), configured closed egress policies (`allow_internet = false`), and verified in-VM `EgressProbe` digest. Checking only the

acknowledgement boolean is not network enforcement.

- Tasks requiring Tier-2 isolation MUST NOT be downgraded silently to Tier 1 when Tier 2 / KVM is unavailable; they are strictly quarantined. Downgrade requires an explicit, authenticated operator risk reassessment override (`aep dispatch --override-tier1-risk-reassessment --reason="..."`), recorded in the decision record with the operator identity.
- Tier-2 capability is derived from the resolved `Backend`, never from host family or `/dev/kvm` alone: `KVMProbe` selects the Cube/KVM candidate, while `Tier2Supported()` reports whether a usable backend exists. Capability and capacity are separate failure reasons.
- Independent re-verification of Tier-2 patches MUST execute in a fresh disposable sandbox whose isolation meets the task `Requirement`. A gVisor-only host may re-verify gVisor-eligible work; it MUST quarantine hardware-floor work before the original run. Tier 1 performs static analysis only (syntax, diff, lint).
- Model inference occurs in the trusted host harness, outside every lane. A lane has no provider credential and cannot originate a model request; lane output is untrusted tool data, never interpolated into a system prompt.
- Model tiering is selected by the host harness: ralph work at `standard` / `measured`; `frontier` / `thorough` reserved for factory runs.
- Receipts bind full-stream `StdoutSha` / `StderrSha`, `Nonce`, `IssuedAt`, `LedgerSeq`, and `KeyID`, and carry an Ed25519 `Signature`. A replayed nonce/sequence, altered output byte, wrong key, or missing signature fails verification.
- Every lane uses `exec.sock` for allowlisted commands and `git.sock` for status/diff/commit. No host `.git` directory is mounted; the linked-worktree `.git` file is masked.
- A task has exactly one ledger identity (the beads ID) across both lanes.

10. Implementation status (as of this document)

BUILT in `develop`

- `cmd/aep` with `version`, `info`, `boot`, `verify`, `supervisor`, `lock-test`.
- `internal/platform/{memory,lock,process}` for Linux, macOS and Windows; `internal/verifier/runner` (allowlist discovery, full-stream SHA-256 with display tails, memory preflight, budgets).
- `Makefile` targets `binaries`, `vm`, `oci`; `scripts/package-vm.sh`, `scripts/package-oci.sh`; `Containerfile`, `docker-compose.yml`.
- Root CI runs `gofmt`, `go vet`, `go test`.

NOT YET BUILT

- `aep boot` today checks only the memory floor and platform; the NeuroStrata/guard/gh-auth/policy checks still live in `scripts/boot-gate.sh`.
- `aep supervisor` today runs boot gate + one verification pass per tick; it does not dispatch lanes or ESF runs.
- Lane coordination remains in `scripts/lane-coord.sh` (Linux `flock`); no Go lane manager, no admission control, no beads adapter, no dispatcher, no ESF client, no Colima profile or compose topology for the substrate.

- The "unified verifier inside the CubeSandbox template" and NeuroStrata hooks in the ESF harness are design intent only.

11. Risks and open questions

Risk / question	Why it matters	Cheapest check
MED KVM inside Colima/VZ requires `--nested-virtualization` — confirmed 2026-11-09; CubeSandbox itself still unvalidated	CubeSandbox microVMs require Linux KVM. On macOS, KVM exists inside the Colima VM *only* with `--nested-virtualization` (confirmed: `/dev/kvm` present, QEMU `accel=kvm` accepted on M3 Max / Colima 0.10.3). Additional local-first backends remain: `libkrun` on macOS (native `vz`) and `gVisor` (`runsc systrap` on KVM-free Linux). Remote cluster is strictly an emergency last resort.	`colima ssh -- ls -l /dev/kvm` in the VM; probe at `aep boot` and record `KVMProbe`. Derive `Tier2Supported()` from the resolved `Backend`, never from the KVM result alone. Next: validate CubeSandbox-in-Colima.
HIGH Model boundary and prompt injection	Hosted inference requires provider credentials. Putting a model relay inside a lane would turn zero network egress into a semantic exfiltration channel and expose provider keys to untrusted code.	Run model inference in the trusted host harness; lane output is untrusted tool data. Canary test: a lane cannot originate a provider request; `exec.sock` rejects shell interpolation. Verify no provider key exists in the lane env.
HIGH CubeSandbox and Temporal are external	ESF discovers an existing CubeSandbox; it never installs one. Neither exists on this host.	`factory doctor` against the target environment.
MED Lane footprints are unmeasured	Admission thresholds depend on real RSS of ralph and ESF lanes across hardware profiles.	Install Colima/Podman, run representative lanes, record peak RSS.
MED Tier-1 isolation is container-only	Lane containers share one kernel; a hostile task in Tier 1 is not contained like Tier 2.	Keep untrusted work routed to Tier 2; enforce quarantine on fallback.
MED Boot parity gap	Moving to `aep boot` without the daemon/guard/policy checks would silently weaken governance versus `boot-gate.sh`.	Port checks with tests before retiring the shell gate.
LOW Singular/plural docs path	This document lives in `docs/plan/`; the repo already has `docs/plans/` and `docs/superpowers/plans/`.	Decide on one location.

12. Review status & PR #133 review remediations

The v1.1 design incorporates the PR #133 review findings (C1–C12). C1 (probe-verified egress), C2 (no silent downgrade), and C3 (host-mediated memory/GitHub relays) are restored or verified at head. C4 and C5 are reopened and fixed in this revision: re-verification is gated by a task `Requirement` rather than an unconditional hardware microVM, and the nano preset is coherent at 2560 + 3584 + 1536 = 7680 MiB. C6–C12 are addressed by the checked admission predicate, host-harness model boundary, host-

side `git.sock`, typed memory, and full-stream signed receipts. Supervisory relays share one `relay.Handler` contract.
